

AUTORABIT JAVA SCREENING MASTER CHEATSHEET

1. BASIC JAVA TEMPLATE

Scanner Version

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        System.out.println(n);

        sc.close();

    }

}
```

Fast Input Version (Important)

```
import java.io.*;
import java.util.*;

public class Main {

    public static void main(String[] args) throws IOException {

        BufferedReader br = new BufferedReader(new
        InputStreamReader(System.in));

        int n = Integer.parseInt(br.readLine());

        StringTokenizer st = new StringTokenizer(br.readLine());

        int[] arr = new int[n];

        for(int i = 0; i < n; i++) {
            arr[i] = Integer.parseInt(st.nextToken());
        }

    }

}
```

```
    }  
  
    System.out.println("Done");  
}  
}
```

2. BASIC LOOPS

For Loop

```
for(int i = 0; i < n; i++) {  
  
}
```

Enhanced For Loop

```
for(int x : arr) {  
    System.out.println(x);  
}
```

While Loop

```
while(n > 0) {  
    n--;  
}
```

3. FUNCTIONS

```
static int add(int a, int b) {  
    return a + b;  
}
```

```
static void printArray(int[] arr) {  
  
    for(int x : arr) {  
        System.out.print(x + " ");  
    }  
}
```

4. ARRAYS

Input Array

```
int n = sc.nextInt();

int[] arr = new int[n];

for(int i = 0; i < n; i++) {
    arr[i] = sc.nextInt();
}
```

Print Array

```
for(int x : arr) {
    System.out.print(x + " ");
}
```

Sort Array

```
Arrays.sort(arr);
```

Reverse Array

```
int l = 0;
int r = arr.length - 1;

while(l < r) {

    int temp = arr[l];
    arr[l] = arr[r];
    arr[r] = temp;

    l++;
    r--;
}
```

Largest Element

```
int max = arr[0];

for(int x : arr) {
```

```
    if(x > max) {  
        max = x;  
    }  
}
```

Second Largest

```
int first = Integer.MIN_VALUE;  
int second = Integer.MIN_VALUE;  
  
for(int x : arr) {  
    if(x > first) {  
        second = first;  
        first = x;  
    }  
    else if(x > second && x != first) {  
        second = x;  
    }  
}
```

5. STRINGS

Input

```
String s = sc.next();  
  
String line = sc.nextLine();
```

Important Methods

```
s.length()  
  
s.charAt(i)  
  
s.substring(a,b)  
  
s.equals(str)  
  
s.toLowerCase()
```

```
s.toUpperCase()
```

```
s.toCharArray()
```

Reverse String

```
String rev = "";  
  
for(int i = s.length()-1; i >= 0; i--) {  
    rev += s.charAt(i);  
}
```

Palindrome

```
String rev = "";  
  
for(int i = s.length()-1; i >= 0; i--) {  
    rev += s.charAt(i);  
}  
  
if(s.equals(rev)) {  
    System.out.println("Palindrome");  
}
```

StringBuilder

```
StringBuilder sb = new StringBuilder(s);  
  
sb.reverse();  
  
System.out.println(sb);
```

6. TYPE CONVERSIONS

String to Int

```
int num = Integer.parseInt(str);
```

Int to String

```
String s = String.valueOf(num);
```

Char to Int

```
int val = ch - '0';
```

Int to Char

```
char ch = (char)(val + '0');
```

7. ARRAYLIST

Declaration

```
ArrayList<Integer> list = new ArrayList<>();
```

Operations

```
list.add(10);  
  
list.get(0);  
  
list.remove(0);  
  
list.size();
```

Traversal

```
for(int x : list) {  
    System.out.println(x);  
}
```

Sort

```
Collections.sort(list);
```

Descending Sort

```
Collections.sort(list, Collections.reverseOrder());
```

8. HASHMAP (VERY IMPORTANT)

Declaration

```
HashMap<Integer,Integer> map = new HashMap<>();
```

Operations

```
map.put(1,10);  
  
map.get(1);  
  
map.containsKey(1);  
  
map.remove(1);
```

SAFEST FREQUENCY TEMPLATE

```
HashMap<Character,Integer> map = new HashMap<>();  
  
for(char c : s.toCharArray()) {  
    map.put(c, map.getOrDefault(c, 0) + 1);  
}
```

Traverse HashMap

```
for(Map.Entry<Character,Integer> e : map.entrySet()) {  
    System.out.println(e.getKey() + " " + e.getValue());  
}
```

9. HASHSET

Declaration

```
HashSet<Integer> set = new HashSet<>();
```

Duplicate Detection

```
for(int x : arr) {  
    if(set.contains(x)) {  
        System.out.println("Duplicate");  
    }  
    set.add(x);  
}
```

10. PRIORITY QUEUE (HEAPS)

Min Heap

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
```

Max Heap

```
PriorityQueue<Integer> pq =  
    new PriorityQueue<>(Collections.reverseOrder());
```

Operations

```
pq.offer(10);  
  
pq.poll();  
  
pq.peek();
```


11. BINARY SEARCH

```
int left = 0;
int right = arr.length - 1;

while(left <= right) {

    int mid = left + (right - left) / 2;

    if(arr[mid] == target) {
        return mid;
    }

    if(arr[mid] < target) {
        left = mid + 1;
    }

    else {
        right = mid - 1;
    }
}

return -1;
```

12. SLIDING WINDOW

```
int maxSum = 0;
int windowSum = 0;

for(int i = 0; i < k; i++) {
    windowSum += arr[i];
}

maxSum = windowSum;

for(int i = k; i < arr.length; i++) {

    windowSum += arr[i] - arr[i-k];

    maxSum = Math.max(maxSum, windowSum);
}
```

13. TWO SUM

```
HashMap<Integer,Integer> map = new HashMap<>();

for(int i = 0; i < arr.length; i++) {

    int target = sum - arr[i];

    if(map.containsKey(target)) {

        System.out.println(map.get(target) + " " + i);
    }

    map.put(arr[i], i);
}
```

14. SORTING

Bubble Sort

```
for(int i = 0; i < n-1; i++) {

    for(int j = 0; j < n-i-1; j++) {

        if(arr[j] > arr[j+1]) {

            int temp = arr[j];

            arr[j] = arr[j+1];

            arr[j+1] = temp;
        }
    }
}
```

15. OOPS RAPID RECALL

4 Pillars

- Encapsulation
- Inheritance
- Polymorphism

- Abstraction

Overloading

Same method name, different parameters

Overriding

Child class redefines parent method

Interface

- supports abstraction
- multiple inheritance support

Abstract Class

- can have abstract + normal methods

this

Current object reference

super

Parent class reference

static

Belongs to class, not object

final

Cannot modify / override / inherit

16. EXCEPTION HANDLING

```
try {  
      
}  
catch(Exception e) {  
      
}  
finally {  
      
}
```

```
}
```

Checked Exception

Compile-time checked

Examples:

- IOException
- SQLException

Unchecked Exception

Runtime

Examples:

- NullPointerException
- ArithmeticException

throw

Explicitly throw exception

throws

Declare exception in method signature

17. IMPORTANT IMPORTS

```
import java.util.*;  
import java.io.*;
```

18. COMMON MISTAKES

- using == instead of .equals()
- forgetting semicolon
- wrong loop condition
- index out of bounds
- Scanner + nextLine confusion
- not checking containsKey
- forgetting edge cases

- not dry running
-

19. HIGH PROBABILITY QUESTION TYPES

Strings

- palindrome
- reverse string
- anagram
- frequency count

Arrays

- largest/smallest
- second largest
- duplicates
- sorting
- two sum

HashMap

- frequency
- counting
- lookup

Sliding Window

- max sum subarray
 - longest substring
-

20. EXAM STRATEGY

1. Solve easiest first
 2. Write brute force first
 3. Then optimize
 4. Dry run sample input
 5. Handle edge cases
 6. Don't panic if stuck
 7. Partial working code > unfinished optimal solution
-

21. FINAL RAPID RECALL BEFORE TEST

Revise:

- loops
- arrays
- strings
- HashMap
- ArrayList
- Binary Search
- Sliding Window
- OOPS definitions
- Exception basics

That alone covers a huge percentage of screening rounds.